

业务逻辑漏洞与越权漏洞安全分析报告

一、概述

本次安全测试针对基于Flask框架开发的用户管理系统，聚焦个人中心信息查询（/profile）、用户余额充值（/recharge）两大核心业务模块。通过源代码审计、动态漏洞复现、风险量化评估等方式，全面挖掘系统存在的未授权访问、水平越权、客户端身份可控、业务逻辑校验缺失等高危安全缺陷，梳理漏洞成因、攻击链路与安全危害，提供可直接部署落地的修复代码、防护策略与安全规范，为系统业务安全加固提供依据。

项目	内容
报告名称	业务逻辑漏洞与越权漏洞安全分析报告
目标系统	用户管理系统（Flask Web 应用）
审计版本	app.py
整体风险等级	高危（Critical）
测试方式	源代码审计、漏洞动态复现、CVSS风险量化、攻击链建模
测试日期	2026-07-10

二、漏洞摘要

经全面审计，系统个人中心与充值业务模块共存在5项安全漏洞，包含4项高危漏洞、1项中危漏洞，覆盖身份认证失效、访问控制失效、业务逻辑校验缺失、客户端可控信任等核心Web安全风险，攻击者可实现批量用户信息窃取、任意用户余额恶意篡改、身份伪造等攻击，严重破坏系统数据保密性与完整性。漏洞汇总如下：

漏洞编号	漏洞名称	风险等级	漏洞状态	核心危害
VULN-AUTH-001	水平越权漏洞 (IDOR不安全对象直接引用)	高危	未修复	普通用户可越权查看任意用户敏感信息
VULN-AUTH-002		高危	未修复	

	接口未授权访问漏洞			未登录用户可批量窃取全站用户数据
VULN-BIZ-001	充值金额无校验 (负值余额操纵)	高危	未修复	恶意扣减任意用户余额，破坏财务数据
VULN-BIZ-002	用户身份完全由客户端可控	高危	未修复	伪造身份操作任意用户业务数据
VULN-BIZ-003	用户余额可被恶意置为负数	中危	未修复	业务数据异常，数据完整性受损

三、漏洞详细分析

3.1 VULN-AUTH-001：水平越权漏洞（IDOR 不安全对象直接引用）

漏洞位置：app.py 第105-118行，/profile 个人资料查询路由

漏洞源码：

代码块

```
1  @app.route("/profile", methods=["GET"])
2  def profile():
3      user_id = request.args.get("user_id", type=int)
4      user_info = None
5      error = None
6      if user_id is not None:
7          user_info = find_user_by_id(user_id)
8          if user_info is None:
9              error = f"未找到 user_id 为 {user_id} 的用户"
10     else:
11         error = "请提供 user_id 参数"
12     return render_template("profile.html", user_info=user_info, error=error)
```

漏洞原理与描述：系统个人资料查询接口完全依赖客户端可控的URL参数 `user_id` 定位查询用户数据，服务端未做任何**身份鉴权与资源所有权校验**。服务端默认信任客户端传入的查询参数，未校验当前登录用户是否拥有目标用户数据的访问权限，存在典型的水平越权（IDOR）漏洞。普通登录用户可通过修改URL中的user_id参数，随意查询系统内其他所有用户的隐私数据。

漏洞复现与攻击场景：攻击者使用普通用户alice账号登录系统后，仅需修改请求参数，即可越权查询管理员及其他所有用户信息：

代码块

```
1  # 登录普通用户账号
```

```
2 curl -X POST -d "username=alice&password=123456" http://target/login
3 # 越权查询管理员资料 (user_id=1)
4 curl http://target/profile?user_id=1
```

攻击结果：成功获取管理员账号的邮箱、手机号、账户余额等核心敏感隐私信息。任意普通用户均可互相越权访问，无任何权限隔离机制。

3.2 VULN-AUTH-002：接口未授权访问漏洞

漏洞描述： /profile 核心敏感接口未配置任何登录认证拦截逻辑，无会话校验机制。未登录、未授权的匿名用户可直接访问接口，通过枚举user_id遍历查询全站所有用户的个人资料，无任何访问门槛与安全拦截。

攻击复现： 无需登录系统，直接发送请求批量遍历用户ID，批量窃取全站用户隐私数据：

代码块

```
1 # 匿名直接查询管理员信息
2 curl http://target/profile?user_id=1
3 # 批量枚举遍历全站用户数据
4 for id in {1..5}; do curl "http://target/profile?user_id=$id"; done
```

安全危害： 攻击者可批量爬取系统所有用户的手机号、邮箱、余额等敏感数据，造成大规模用户信息泄露，引发社工攻击、隐私泄露、数据倒卖等安全风险。

3.3 VULN-BIZ-001：充值金额无校验（负值充值与余额操纵）

漏洞位置： app.py 第121-130行， /recharge 余额充值路由

漏洞源码：

代码块

```
1 @app.route("/recharge", methods=["POST"])
2 def recharge():
3     user_id = request.form.get("user_id", type=int)
4     amount = request.form.get("amount", type=float, default=0)
5     user = find_user_by_id(user_id)
6     if user:
7         user["balance"] = user["balance"] + amount
8     return redirect(f"/profile?user_id={user_id}")
```

漏洞原理与描述： 充值业务核心逻辑存在严重的业务校验缺失，服务端对客户端传入的 `amount` 充值金额参数**未做正负值校验、数值范围校验**。程序直接将传入数值与用户余额做累加计算，攻击者可传入负数金额，实现「反向充值」，恶意扣减任意用户账户余额。

攻击复现：目标用户alice初始余额100，攻击者构造负数充值请求，恶意扣减余额：

代码块

```
1 curl -X POST -d "user_id=2&amount=-500" http://target/recharge
```

攻击结果：alice账户余额由100变为-400，余额被恶意篡改，业务数据完全失控。

组合攻击链：结合越权漏洞，攻击者可先越权获取管理员user_id，再通过负值充值接口恶意清空、扣减管理员账户余额，实现精准业务破坏。

3.4 VULN-BIZ-002：用户身份完全由客户端控制

漏洞描述：系统所有核心业务接口的用户身份标识（user_id）均由客户端可控参数传入，/profile通过URL参数获取、/recharge通过表单参数获取，服务端完全信任客户端数据，**未从服务端Session会话读取真实登录身份**。攻击者可随意篡改user_id参数，伪造身份操作任意用户的业务数据，造成身份伪造、越权操作等一系列高危风险。

攻击示例：普通用户可直接修改参数操作管理员账户：

代码块

```
1 curl -X POST -d "user_id=1&amount=-99999" http://target/recharge
```

攻击结果：管理员账户余额被恶意扣减99999，无任何权限拦截与身份校验。

3.5 VULN-BIZ-003：余额可被任意减为负数

漏洞描述：系统余额更新逻辑仅做简单累加运算，未对运算后的余额数值做合法性校验。结合负值充值漏洞，攻击者可将任意用户的账户余额篡改至负数，导致系统财务数据错乱、业务数据异常，破坏数据完整性，影响平台账务统计与业务合规性。

四、CVSS 3.1 风险量化评估

采用行业通用CVSS 3.1评分标准，从攻击向量、攻击复杂度、权限要求、用户交互、影响范围及机密性、完整性、可用性（CIA三元组）全方位量化漏洞风险，评分精准匹配漏洞危害等级：

漏洞编号	攻击向量	复杂度	权限	交互	范围	保密性	完整性	可用性	CVSS评分
VULN-AUTH-001	网络	低	低	无	未改变	高	无	无	6.5（中危）

VULN-AUTH-002	网络	低	无	无	未改变	高	无	无	7.5（高危）
VULN-BIZ-001	网络	低	无	无	未改变	无	高	高	9.1（高危）
VULN-BIZ-002	网络	低	低	无	未改变	高	高	高	9.0（高危）
VULN-BIZ-003	网络	低	低	无	未改变	无	高	低	6.5（中危）

五、完整攻击链分析

5.1 批量用户信息泄露攻击链

匿名攻击者 → 无需登录认证 → 枚举遍历user_id → 批量调用/profile接口 → 获取全站用户手机号、邮箱、余额等敏感数据 → 用于社工诈骗、数据倒卖、账户劫持

5.2 越权身份伪造+余额恶意篡改攻击链

普通用户登录系统 → 利用水平越权漏洞查询管理员user_id → 伪造客户端参数调用/recharge接口 → 传入负值金额 → 恶意扣减管理员/任意用户余额 → 造成平台账务数据错乱、业务受损

六、分层落地修复方案

针对认证失效、访问控制缺失、业务逻辑缺陷三大核心问题，采用「装饰器认证+服务端身份绑定+权限校验+业务参数强校验」多层防护体系，所有修复代码可直接替换原业务代码，零改造门槛、全覆盖漏洞风险。

6.1 通用登录认证装饰器（修复未授权访问）

统一封装登录校验装饰器，所有敏感接口强制校验会话，杜绝匿名访问：

代码块

```
1  from functools import wraps
2  from flask import session, redirect
3
4  # 登录认证拦截装饰器
5  def login_required(f):
6      @wraps(f)
7      def decorated_function(*args, **kwargs):
8          if "username" not in session:
9              return redirect("/login")
10         return f(*args, **kwargs)
```

```
11         return decorated_function
```

6.2 资源所有权权限校验（修复水平越权）

区分管理员与普通用户权限，普通用户仅可访问自身数据，管理员拥有全局查询权限：

代码块

```
1  def check_ownership(target_user_id):
2      # 从服务端会话获取真实登录身份
3      current_user_id = session.get("user_id")
4      current_role = session.get("role")
5      # 管理员放行所有访问
6      if current_role == "admin":
7          return True
8      # 普通用户仅允许访问自身资源
9      return current_user_id == target_user_id
```

6.3 修复后安全的个人中心接口

代码块

```
1  @app.route("/profile", methods=["GET"])
2  @login_required
3  def profile():
4      user_id = request.args.get("user_id", type=int)
5      user_info = None
6      error = None
7      if user_id is not None:
8          # 强制校验资源所有权，防御越权
9          if not check_ownership(user_id):
10             error = "权限不足，无权访问该用户资料"
11         else:
12             user_info = find_user_by_id(user_id)
13             if user_info is None:
14                 error = f"未找到 user_id 为 {user_id} 的用户"
15         else:
16             error = "请提供 user_id 参数"
17     return render_template("profile.html", user_info=user_info, error=error)
```

6.4 修复后安全的充值业务接口

核心优化：身份从服务端Session获取、金额正负/上限双校验、杜绝客户端篡改身份、防止负值扣费、防止余额负数异常：

```
代码块
1 app.route("/recharge", methods=["POST"])
2 @login_required
3 def recharge():
4     # 服务端会话获取真实用户ID, 杜绝客户端身份伪造
5     user_id = session.get("user_id")
6     amount = request.form.get("amount", type=float, default=0)
7     user_info = find_user_by_id(user_id)
8
9     # 1. 金额正负合法性校验
10    if amount <= 0:
11        error = "充值金额必须大于0, 禁止负值充值"
12        return render_template("profile.html", user_info=user_info,
13                                error=error)
14
15    # 2. 充值金额上限约束, 防止超大金额异常
16    MAX_RECHARGE = 10000
17    if amount > MAX_RECHARGE:
18        error = f"单次充值金额不可超过{MAX_RECHARGE}元"
19        return render_template("profile.html", user_info=user_info,
20                                error=error)
21
22    # 3. 执行余额更新
23    if user_info:
24        user_info["balance"] = user_info["balance"] + amount
25
26    return redirect(f"/profile?user_id={user_id}")
```

6.5 漏洞修复方案对照表

修复措施	修复对应漏洞	改造难度	防护效果
添加 login_required 认证装饰器	未授权访问漏洞	极低	彻底拦截匿名访问
新增资源所有权校验函数	水平越权漏洞	极低	完全杜绝跨用户越权访问
金额正负+上限双校验	负值充值、余额操纵	极低	杜绝恶意扣费与异常充值
Session 服务端获取用户身份	客户端身份可控漏洞	极低	彻底杜绝身份伪造

七、OWASP Top 10 2021 权威映射

结合OWASP官方2021版Web安全风险标准，完成漏洞权威归类，贴合行业安全规范：

漏洞编号	OWASP Top 10 2021 分类	风险说明
VULN-AUTH-001	A01:2021 访问控制失效	未做资源权限隔离，存在水平越权数据泄露
VULN-AUTH-002	A01:2021 访问控制失效	接口无认证机制，匿名用户可访问敏感资源
VULN-BIZ-001	A04:2021 不安全的设计	核心业务逻辑缺少必要参数校验，属于设计层面缺陷
VULN-BIZ-002	A01:2021 访问控制失效	身份认证逻辑设计错误，信任客户端可控数据
VULN-BIZ-003	A04:2021 不安全的设计	业务结算逻辑无数据合法性校验，导致数据异常

八、安全审计 Checklist（标准化自查清单）

- ☒ 所有敏感业务接口已添加登录会话认证拦截
- ☒ 所有用户私有资源已配置所有权权限校验，杜绝越权
- ☒ 核心业务身份标识统一从服务端Session获取，不信任客户端参数
- ☒ 金额类参数已配置正负校验、上下限范围校验
- ☒ 账务结算结果已做合法性校验，杜绝负数异常数据
- ☒ 前端隐藏字段不再作为身份校验依据，彻底杜绝身份伪造
- ☒ 所有未授权敏感接口已全部加固拦截

九、漏洞修复优先级与整改建议

漏洞编号	优先级	整改时限	修复难度	影响说明
VULN-AUTH-001	P0 紧急修复	24小时内	低	全站用户隐私数据可被任意越权查询，泄露风险极高
VULN-AUTH-002	P0 紧急修复	24小时内	低	无任何访问门槛，匿名批量窃取用户数据
VULN-BIZ-001	P0 紧急修复	24小时内	低	可恶意篡改任意用户余额，破坏核心业务数据

VULN-BIZ-002	P1 尽快修复	1周内	低	身份可被伪造，是所有越权业务漏洞的根源
VULN-BIZ-003	P1 尽快修复	1周内	低	导致账务数据异常，影响业务合规性

十、总结与安全最佳实践

10.1 漏洞核心总结

本次审计发现系统个人中心、充值模块共存在5项安全漏洞，其中3项P0级高危漏洞，可直接造成全站用户信息泄露、核心账务数据被恶意篡改。所有漏洞根源可归纳为三点：一是**认证体系缺失**，敏感接口无登录拦截；二是**访问控制完全失效**，未做用户资源权限隔离；三是**业务设计不安全**，过度信任客户端数据，缺失参数合法性强校验。

10.2 核心极简修复口诀（四行核心安全代码）

代码块

```
1  # 1. 所有敏感接口强制登录校验
2  @login_required
3  # 2. 核心业务身份从服务端会话获取，不信任客户端
4  user_id = session.get("user_id")
5  # 3. 金额参数强制合法性校验
6  if amount <= 0: return "充值金额不合法"
7  # 4. 私有资源强制所有权校验
8  if not check_ownership(user_id): return "权限不足"
```

10.3 长期业务安全最佳实践

1. 服务端永不信任客户端：所有身份、权限、业务核心参数必须由服务端会话管控，禁止前端传参控制身份；
2. 严格遵循最小权限原则：普通用户仅可操作自身数据，管理员权限单独隔离管控；
3. 业务参数强校验：金额、数量、ID等核心参数必须做正负、范围、合法性校验；
4. 敏感接口统一认证：所有隐私查询、资金操作接口统一添加认证与权限拦截；
5. 建立代码自查机制：上线前完成权限、认证、参数校验标准化自查，规避逻辑漏洞。

（注：部分内容可能由 AI 生成）